অধ্যায় ৪

Loops এবং Iteration: Repeated Work থেকে Cart Analyzer

শেখার লক্ষ্য

এই অধ্যায়ের শেষে পাঠক `for`, `while`, `do...while`, `for...of`, `for...in` এবং Array Iteration বুঝতে পারবে; `break`, `continue`, Nested Loop, Complexity ও Infinite Loop-এর ঝুঁকি চিনবে; এবং একটি বাস্তব **Cart Analyzer** বানাবে।

৪.১ Loop কেন দরকার?

ধরা যাক, একটি অনলাইন দোকানে ১০০টি পণ্য আছে। প্রতিটির দাম আলাদাভাবে যোগ করা অযৌক্তিক। Loop একই নিয়ম একাধিক Data Item-এর ওপর প্রয়োগ করতে সাহায্য করে।

```javascript
const prices = [120, 250, 90];
let total = 0;
for (let i = 0; i < prices.length; i++) {
  total += prices[i];
}
console.log(total); // 460
```

এখানে `i = 0` হলো Initialization, `i < prices.length` হলো Condition এবং `i++` হলো Update। Condition মিথ্যা হলে Loop থামে। **Off-by-one Error** এড়াতে Index-এর ক্ষেত্রে `< array.length` ব্যবহার করো; `<=` নয়।

৪.২ `for` Loop: নির্দিষ্ট Range

```javascript
for (let n = 1; n <= 5; n++) console.log(n);

// শুধু জোড় সংখ্যা
for (let n = 2; n <= 10; n += 2) console.log(n);

// Reverse countdown
for (let n = 5; n >= 1; n--) console.log(n);
```

কোনো Collection-এর Index দরকার হলে `for` উপযুক্ত। কিন্তু শুধু Value প্রয়োজন হলে `for...of` সাধারণত বেশি পাঠযোগ্য।

৪.৩ `while` এবং `do...while`

`while` আগে Condition পরীক্ষা করে। `do...while` অন্তত একবার Body চালায়।

```javascript
let attempts = 0;
while (attempts < 3) {
  attempts++;
  console.log(`Attempt ${attempts}`);
}

let answer;
do {
  answer = "yes"; // বাস্তবে Input থেকে আসতে পারে
  console.log("Prompt shown once");
} while (answer !== "yes");
```

সতর্কতা: Condition পরিবর্তিত না হলে Infinite Loop হতে পারে। Browser Main Thread-এ এমন Loop UI আটকে দিতে পারে।

৪.৪ `break` ও `continue`

`break` পুরো Loop বন্ধ করে; `continue` বর্তমান Iteration বাদ দিয়ে পরের Iteration-এ যায়।

```javascript
const values = [10, -1, 0, 25, 40];
for (const value of values) {
  if (value < 0) continue; // Negative উপেক্ষা
  if (value === 0) break;  // Sentinel পেলে থামো
  console.log(value);      // শুধু 10
}
```

এগুলো বেশি ব্যবহার করলে Control Flow জটিল হতে পারে। প্রয়োজনে আলাদা Function বানাও।

৪.৫ `for...of` বনাম `for...in`

`for...of` Iterable-এর **Value** দেয়। `for...in` Object-এর Enumerable **Property Keys** দেয়; Array Iteration-এর জন্য সাধারণত ব্যবহার করা উচিত নয়।

```javascript
const destinations = ["Dhaka", "Sylhet", "Cox's Bazar"];
for (const destination of destinations) console.log(destination);

const product = { name: "Notebook", price: 180 };
for (const key in product) {
  if (Object.hasOwn(product, key)) console.log(key, product[key]);
}
```

`Object.keys()`, `Object.values()`, `Object.entries()` ব্যবহার করলে Object-এর Own Properties নিয়ে কাজ আরও পরিষ্কার হয়।

```javascript
for (const [key, value] of Object.entries(product)) {
  console.log(`${key}: ${value}`);
}
```

৪.৬ Nested Loops ও Complexity

একটি Loop-এর ভিতরে আরেকটি Loop হলে মোট কাজ বাড়ে।

```javascript
for (let row = 1; row <= 3; row++) {
  let line = "";
  for (let col = 1; col <= 4; col++) line += "* ";
  console.log(line);
}
```

উপরের Code মোট ১২ বার Inner Body চালায়। Input Size `n` হলে একটি সাধারণ Linear Loop **O(n)**; দুটি Independent Full Nested Loop প্রায় **$O(n^2)$**। Complexity একটি Growth Model, নির্দিষ্ট Runtime নয়।

Pattern	আনুমানিক Time Complexity	উদাহরণ
একটি স্থির কাজ	O(1)	Array Length পড়া
একবার সব Item দেখা	O(n)	Cart Total
দুটি পূর্ণ Nested Loop	$O(n^2)$	সব Pair তুলনা
Input প্রতি ধাপে অর্ধেক	O(log n)	Binary Search (Sorted Data)

৪.৭ Array Methods-ও Iteration করে

```javascript
const numbers = [2, 4, 6];
console.log(numbers.map(n => n * 2));       // [4, 8, 12]
console.log(numbers.filter(n => n > 3));    // [4, 6]
console.log(numbers.reduce((s, n) => s + n, 0)); // 12
```

`map` নতুন Array তৈরি করে; `filter` বাছাই করে; `reduce` Accumulator তৈরি করে। কোনটি ব্যবহার করবে তা কাজের উদ্দেশ্য ও Readability-এর ওপর নির্ভর করে। `forEach` থেকে `break` করা যায় না; Early Exit দরকার হলে `for...of` বা `some` / `find` বিবেচনা করো।

৪.৮ Practical Project — Cart Analyzer

সমস্যা: একটি Cart-এ Product Name, Unit Price এবং Quantity আছে। আমাদের দরকার Line Subtotal, Total BDT, মোট Unit, সবচেয়ে বড় Line Amount এবং একটি Threshold-এর বেশি Subtotal-যুক্ত পণ্যের তালিকা।

Project Files:

```
source-code/chapter-04/
├── index.html
├── style.css
├── cart.js
├── app.js
└── cart.test.js
```

Data Contract: `[{ name: "Notebook", price: 120, quantity: 3 }]`। এখানে Price পূর্ণসংখ্যা BDT। Production Finance Application-এ Minor Units বা Decimal Strategy আলাদাভাবে নির্ধারণ করতে হবে।

Core Algorithm (`cart.js`)

JAVASCRIPT

```javascript
function analyzeCart(items, threshold = 5000) {
  validateCart(items);
  let total = 0;
  let units = 0;
  let highest = null;
  const rows = [];
  for (let i = 0; i < items.length; i++) {
    const item = items[i];
    const subtotal = item.price * item.quantity;
    if (!Number.isSafeInteger(subtotal) || !Number.isSafeInteger(total + subtotal)) {
      throw new RangeError("Amount exceeds safe integer range");
    }
    total += subtotal;
    units += item.quantity;
    const row = { name: item.name.trim(), price: item.price, quantity: item.quantity, subtotal
};
    rows.push(row);
    if (highest === null || subtotal > highest.subtotal) highest = row;
  }
  const qualifying = [];
  for (const row of rows) if (row.subtotal >= threshold) qualifying.push(row.name);
  return { rows, total, units, distinctItems: rows.length, highest, qualifying };
}
```

উপরের Snippet-এ `validateCart` একটি আলাদা Function। **সম্পূর্ণ Runnable Implementation** `source-code/chapter-04/cart.js`-এ আছে। এটি Type Validation, Quantity Range এবং Integer Overflow পরীক্ষা করে। `app.js` DOM Events নিয়ন্ত্রণ করে, `index.html` UI তৈরি করে, আর `style.css` Layout দেয়।

Browser-এ চালানোর নিয়ম: একই Folder-এর `index.html` খুলে Product যোগ করো। নতুন Item যোগ হলে Summary ও List Update হবে। Input-এর Error Message UI-তে দেখাবে। `textContent` ব্যবহার করা হয়েছে, যাতে User-Provided Name-কে HTML হিসেবে Execute করা না হয়।

Node.js Test:

TERMINAL

```
cd source-code/chapter-04
node --test cart.test.js
```

Dry Run

Product	Price	Quantity	Subtotal
Pen	50	2	100
Book	300	3	900
Total		**5 units**	**1000 BDT**

`highest.name` হবে `"Book"`। Threshold `500` হলে `qualifying` হবে `["Book"]`। একটি Pass-এ Main Summary তৈরির Time Complexity O(n); Extra Rows রাখার Space Complexity O(n)।

৪.৯ Common Mistakes এবং Debugging

1. `i <= array.length` ব্যবহার করলে শেষ Iteration-এ `undefined` আসতে পারে।
2. `while`-এ Counter Update না করলে Infinite Loop হয়।
3. `for...in` দিয়ে Array ঘুরলে Key String হয় এবং Inherited Enumerable Property-ও আসতে পারে।
4. `forEach(async () => ...)` ব্যবহার করে বাইরের Code Automatically Await করে না। Sequential Async কাজের জন্য `for...of` + `await` ব্যবহার করা যায়।
5. User Input-এর Number Conversion ছাড়া `+` ব্যবহার করলে String Concatenation হতে পারে।
6. Nested Loop-এ একই Counter Variable Reuse করলে Logic নষ্ট হতে পারে।

৪.১০ Exercises

Exercise 1: `for` ব্যবহার করে ১–১০-এর যোগফল বের করো।

Exercise 2: `while` দিয়ে ৫ থেকে ১ পর্যন্ত Countdown লেখো।

Exercise 3: `[3, 8, 11, 16, 19]` থেকে শুধু জোড় সংখ্যা Print করো।

Exercise 4: একটি Array-তে প্রথম `0` পাওয়ার আগে পর্যন্ত Positive Values যোগ করো।

Exercise 5: `[{name:"A",price:10,quantity:2}, {name:"B",price:15,quantity:3}]`-এর Total বের করো।

Exercise 6: Nested Loop দিয়ে ৩×৩ Multiplication Table তৈরি করো।

Exercise 7 (Challenge): Cart Analyzer-এ ১০% Discount যোগ করো, কিন্তু ১০০০ BDT-এর কম Total হলে Discount নয়। Integer Amount নিয়ে Rounding Rule লিখে দাও।

Exercise Solutions

```javascript
// 1
let sum = 0;
for (let i = 1; i <= 10; i++) sum += i;
console.log(sum); // 55

// 2
let n = 5;
```

```
while (n >= 1) console.log(n--);

// 3
for (const value of [3, 8, 11, 16, 19]) {
  if (value % 2 === 0) console.log(value); // 8, 16
}

// 4
let result = 0;
for (const value of [5, 8, 0, 20]) {
  if (value === 0) break;
  if (value > 0) result += value;
}
console.log(result); // 13

// 5
const cart = [{ name: "A", price: 10, quantity: 2 }, { name: "B", price: 15, quantity: 3 }];
let cartTotal = 0;
for (const item of cart) cartTotal += item.price * item.quantity;
console.log(cartTotal); // 65

// 6
for (let r = 1; r <= 3; r++) {
  let line = "";
  for (let c = 1; c <= 3; c++) line += `${r * c} `;
  console.log(line.trim());
}

// 7 — Whole BDT, discount rounded to nearest BDT
function discountedTotal(total) {
  if (!Number.isSafeInteger(total) || total < 0) throw new TypeError("Invalid total");
  if (total < 1000) return total;
  return total - Math.round(total * 0.1);
}
console.log(discountedTotal(1250)); // 1125
```

৪.১১ Knowledge Check — ১০টি MCQ

1. `for` Loop-এর Update অংশ সাধারণত কখন চলে? (A) Body-এর আগে (B) Body-এর পরে (C) একবারই (D) কখনো না
2. `do...while` সর্বনিম্ন কতবার Body চালায়? (A) 0 (B) 1 (C) 2 (D) অসীম
3. `break` কী করে? (A) Loop বন্ধ করে (B) শুধু একটি Iteration Skip (C) Function বানায় (D) Error দেয়
4. `continue` কী করে? (A) Loop বন্ধ (B) বর্তমান Iteration Skip (C) Array Sort (D) Return
5. Array Value পেতে সাধারণত কোনটি? (A) `for...in` (B) `for...of` (C) `switch` (D) `typeof`
6. `i < arr.length` কেন গুরুত্বপূর্ণ? (A) Memory Delete (B) Valid Index Range (C) Sort (D) Encrypt
7. একটি Linear Scan-এর Time Complexity সাধারণত কী? (A) O(1) (B) O(log n) (C) O(n) (D) $O(n^3)$
8. দুটি পূর্ণ Nested Loop সাধারণত কী Complexity? (A) $O(n^2)$ (B) O(1) (C) O(log n) (D) O(n!)
9. `map()` সাধারণত কী Return করে? (A) নতুন Array (B) Boolean (C) HTML (D) Promise সবসময়
10. `forEach`-এর Callback থেকে `break` ব্যবহার করা যায়? (A) হ্যাঁ (B) না (C) শুধু Browser-এ (D) শুধু Node-এ

Answer Key ও Explanation: 1-B (Body শেষে Update), 2-B (Condition পরে পরীক্ষা), 3-A (Loop Exit), 4-B (পরের Iteration), 5-B (Iterable Values), 6-B (শেষ Index হলো length−1), 7-C (প্রতি Item একবার), 8-A (n×n), 9-A (Mapped Array), 10-B (`forEach` Loop Statement নয়)।

৪.১২ অধ্যায়ের সারসংক্ষেপ

Loop হলো Repeated Data Processing-এর ভিত্তি। সঠিক Loop বেছে নেওয়া, Termination Condition ঠিক রাখা, Input Validation এবং Complexity বোঝা—সবই Production-Ready JavaScript-এর গুরুত্বপূর্ণ দক্ষতা। পরের অধ্যায় **Functions**-এ আমরা এই Repeated Logic-কে Reusable Units-এ ভাগ করব।